# Noir Atelier — Agentic System Architecture & Implementation Plan

Oct 29, 2026 · @Arffy technologies

## System Vision

Noir Atelier's agentic system transforms a traditional e-commerce platform into an autonomous, intelligent retail operation. Instead of hard-coded workflows, specialized AI agents collaborate to handle real-world business problems: understanding customer intent, recommending products, managing complex orders, fulfilling returns, and supporting customers—all with human oversight where needed.

### Why Agentic Architecture Matters

Traditional e-commerce systems are brittle. A customer's intent is forced into fixed categories (search, filter, recommend). If they ask a question, a chatbot tries to parse it. If an order breaks, a human manually investigates.

Agentic systems are different. They reason, adapt, and coordinate:

- **Reasoning**: Agents understand context, make decisions, and adjust strategy based on outcomes
- **Tool integration**: Agents call APIs, databases, and business logic as needed
- **Coordination**: Multiple specialized agents work together on complex tasks
- **Learning**: Agent interactions create logs, metrics, and feedback for continuous improvement

For Noir Atelier, this means:

- A customer can ask natural questions ("show me black shirts under ₹1,500"), and the shopping agent understands intent without rigid search syntax
- An order with a damaged item triggers a remedies agent that knows the law, the policy, and decides on replacement, repair, or refund
- A compliance agent prevents publication of unverified claims and flags tax, privacy, and consumer-protection issues

- A support agent handles complex scenarios ("I ordered the wrong size, the exchange doesn't fit either, and I'm now outside the return window") with empathy and legal grounding

### Core Principles

**1. Real Business Data** Agents operate on actual inventory, pricing, orders, and policies—not mocked values. Every decision is grounded in truth.

**2. Human Oversight** Agents have clear escalation paths. High-stakes decisions (refunds, policy exceptions, sensitive complaints) route to humans.

**3. Legal Compliance** India's e-commerce rules, GST, consumer protection, and data privacy are baked into agent logic. Agents refuse to violate the law to make a sale.

**4. Transparency** Customers and team members understand why an agent made a decision. Logs, audit trails, and explainability matter.

**5. Scalability** From 100 to 100,000 orders per week, the system remains reliable. Tool calls are cached, agents batch work, and nothing scales worse than $O(n^2)$.

**6. Reliability** Agent failures degrade gracefully. Timeouts, API errors, and edge cases are handled explicitly. The system never silently fails.

## System Architecture: Multi-Layer, Multi-Agent Design

### Conceptual Architecture

The Noir Atelier agentic system is built on three layers:

**Presentation Layer** — Customer-facing interfaces (web, mobile, email, SMS, voice) where users initiate requests

**Agent Orchestration Layer** — Intent detection, agent routing, tool calling, approval workflows, and response synthesis

**Operational Layer** — Databases (inventory, orders, customers), integrations (payment, shipping, tax), and real-time data sources

### Agent Network

Specialized agents handle distinct responsibilities:

**Catalog Agent** — Product discovery. Understands search intent, filters, recommendations, inventory availability. Reads: catalog, inventory. Writes: none (read-only).

**Shopping Agent** — Cart management, price calculation, discounts, bundle logic. Reads: catalog, inventory, pricing rules. Writes: cart state, wish lists.

**Order Agent** — Checkout orchestration, order creation, payment routing, confirmation emails. Reads: cart, customer data, shipping rules. Writes: orders, payment records.

**Payment Agent** — Payment processing, reconciliation, refund logic, fraud detection. Reads: orders, payment methods. Writes: payment transactions, refund records.

**Fulfillment Agent** — Shipping integration, tracking, delivery updates, logistics coordination. Reads: orders, shipping rules, courier APIs. Writes: shipment records, tracking updates.

**Returns Agent** — Exchange and refund logic, policy enforcement, legal remedies, damage assessment. Reads: orders, returns policy, customer data. Writes: return records, refund/exchange authorizations.

**Support Agent** — Customer inquiries, complaint handling, escalation triage, knowledge base access. Reads: orders, customer history, policies, FAQs. Writes: support tickets, resolution notes.

**Compliance Agent** — Policy enforcement, tax calculation, data privacy, legal review, publication gates. Reads: all configuration, policies, business data. Writes: compliance flags, policy approvals.

**Admin Agent** — Configuration management, business rule updates, manual order handling, reporting. Reads: all operational data. Writes: business configuration, manual adjustments.

## Data Flow Architecture

```
Customer Requests
    ↓
API Gateway (auth, rate limiting, request validation)
    ↓
Intent Classifier (NLP → determine which agent(s) needed)
    ↓
Agent Orchestrator (route to appropriate agent(s), manage state)
    ↓
Agent Network (parallel or sequential execution)
```

```
    ├─ Catalog Agent → Product DB
    ├─ Shopping Agent → Cart State, Inventory
    ├─ Order Agent → Order DB, Payment Agent
    ├─ Fulfillment Agent → Shipping APIs
    ├─ Returns Agent → Order DB, Policy Engine
    ├─ Support Agent → Ticket System, Knowledge Base
    ├─ Compliance Agent → Policy Validator (gate publication)
    └─ Admin Agent → Configuration, Manual Overrides
    ↓
Response Synthesis (format response, include explanations, escalation notes)
    ↓
Customer-Facing Response (API response, email, SMS, web page)
    ↓
Audit Log (all agent decisions, tool calls, tool results, human actions)
```

## Technology Stack (2026 Standard)

### Frontend

- Next.js 15 with TypeScript for web/PWA
- React Native for mobile app
- Real-time updates via WebSocket/Server-Sent Events

### Backend Services

- FastAPI (Python) or Node.js (TypeScript) for agent orchestration
- Service-oriented architecture: catalog service, order service, fulfillment service, etc.
- Message queue (RabbitMQ, Redis, AWS SQS) for async jobs
- Background workers for notifications, reconciliation, scheduled tasks

### Data Layer

- PostgreSQL 16+ for transactional data (customers, orders, products)
- Redis for caching, sessions, real-time data
- Vector DB (Pinecone, Qdrant) for product embeddings, semantic search
- Elasticsearch for full-text search, analytics
- S3-compatible object storage for images, documents

### AI & Agents

- Claude API (multi-model: Opus 5.5 for reasoning, Sonnet 5.5 for speed, Haiku 4.5 for cost)

- LangGraph for agent state management and orchestration
- Tool calling with structured schemas (OpenAPI/JSON Schema)
- Prompt engineering with version control

**Integrations**

- Payment gateway (Razorpay, PayU, or Stripe for India)
- Shipping APIs (Shiprocket, Easypost, or partner couriers)
- SMS/Email (Twilio, AWS SES)
- Tax computation (Cleartax or custom GST engine)
- Analytics (Segment, Mixpanel, or Metabase)

**Infrastructure**

- Kubernetes (AWS EKS, GCP GKE, or Digital Ocean) for scalability
- Terraform/Pulumi for IaC
- GitHub Actions for CI/CD
- Datadog/New Relic for monitoring, logging, alerting
- Prometheus for metrics

**Security**

- OAuth 2.0 / JWT for authentication
- role-based access control (RBAC) for authorization
- TLS 1.3 for all communications
- End-to-end encryption for sensitive data
- Regular penetration testing and security audits
- Compliance: GDPR, India's DPDP Act 2023, RBI guidelines

## Deployment Model

**Development**: Containerized services, mock data, simulated payment/shipping

**Staging**: Production-like environment, real integrations with test credentials, full test suite

**Production**: Multi-region deployment, auto-scaling, failover, zero-downtime deployments

**Agent Reasoning Quality**: Evaluated via metrics (success rate, customer satisfaction, compliance violations) and continuous prompt engineering

# Agent Specifications

## 1. Catalog Agent

**Responsibility**: Product discovery, search, filtering, recommendations, inventory status.

**Tools**

- `search_products(query, filters, page)` → Returns matching products with images, descriptions, prices, stock status
- `get_product_details(product_id)` → Full product spec: colors, sizes, fabric, care instructions, reviews
- `get_inventory(product_id, variant)` → Real-time stock count, reorder status
- `get_recommendations(customer_id, context)` → Personalized recommendations based on history and behavior
- `search_by_style(style_query)` → Semantic search using embeddings (e.g., "minimalist black shirt for office")

**Decision Logic**

- Block searches with clearly misleading queries
- Return "not available" for out-of-stock items; suggest alternatives
- Use semantic search when keyword search fails
- Prioritize bestsellers, highest-rated, price-appropriate items in recommendations

**Escalation

- Customer requests reviews of products (pass to Support Agent)
- Questions about product sourcing, ethics, sustainability (pass to Compliance Agent)

## 2. Shopping Agent

**Responsibility**: Cart management, quantity handling, pricing calculation, discounts, bundling.

**Tools**

- `add_to_cart(customer_id, product_id, variant, quantity)` → Validates availability, adds to cart
- `remove_from_cart(customer_id, item_id)` → Removes item

- `apply_coupon(customer_id, coupon_code, cart_total)` → Validates, applies discount
- `calculate_total(customer_id)` → Subtotal, applicable GST, shipping charge, discounts, final total
- `get_shipping_estimate(cart_contents, delivery_address)` → Shipping cost, delivery estimate
- `validate_cart(customer_id)` → Checks all items still in stock, prices haven't changed

**Decision Logic**

- Block out-of-stock items from being added
- Show real-time price updates if product price changes
- Calculate GST based on product category (clothing GST rates vary)
- Apply free shipping threshold logic (₹1,999+ = free shipping, else ₹99)
- Never show artificially low prices or false discounts

**Escalation**

- Customer requests bulk discounts (pass to Admin Agent)
- Complex discount or bundling questions (pass to Support Agent)

## 3. Order Agent

**Responsibility**: Order creation, checkout flow, payment initiation, order confirmation.

**Tools**

- `create_order(customer_id, cart_items, shipping_address, billing_address)` → Creates order record with unique ID
- `get_payment_methods(customer_id)` → Available payment options (UPI, card, net banking, bank transfer)
- `initiate_payment(order_id, amount, method)` → Calls Payment Agent, returns payment reference
- `confirm_order(order_id, payment_status)` → Finalizes order, sends confirmation email
- `get_order_status(order_id)` → Current order state (pending payment, confirmed, dispatched, delivered, returned)

**Decision Logic**

- Validate shipping address (complete, valid PIN code, serviceable location)

- Create order only after payment confirmation
- Don't allow cash-on-delivery (disabled per business config)
- Generate order confirmation with all terms (shipping, returns policy, contact)
- Prevent duplicate orders (same cart, same customer, within 5 minutes)

**Escalation**

- Customer cancels before payment (allow)
- Customer cancels after payment (pass to Returns Agent)
- Unusual payment method or amount (pass to Payment Agent)
- Address concerns (pass to Support Agent)

## 4. Payment Agent

**Responsibility**: Payment processing, verification, reconciliation, refunds, fraud detection.

**Tools**

- `process_payment(order_id, amount, method, credentials_or_reference)` → Submits to payment gateway, returns status
- `verify_payment(order_id, transaction_ref)` → Confirms payment received via bank reconciliation
- `process_refund(order_id, amount, reason)` → Initiates refund to original payment method
- `get_payment_status(order_id)` → Current payment state (pending, confirmed, failed, refunded)
- `detect_fraud(order_data, customer_history)` → Flags suspicious transactions

**Decision Logic**

- Never store card credentials locally; use payment gateway tokenization
- Verify bank transfers manually (receipt upload, account match)
- Timeout waiting for payment (30 min); release reserved inventory
- Retry failed payments once automatically; then ask customer
- Flag orders from new customers with high-value purchases
- Refunds go to original payment method, not store credit

**Escalation**

- Payment disputes or chargebacks (pass to Admin Agent)

- Refund authorization requests (pass to Returns Agent)
- Customer payment problems (pass to Support Agent)

## 5. Fulfillment Agent

**Responsibility**: Shipping arrangement, tracking, delivery status, coordination with couriers.

**Tools**

- `create_shipment(order_id, items, destination_address)` → Books courier, gets AWB number
- `get_tracking(tracking_number)` → Real-time tracking from courier API
- `get_delivery_estimate(tracking_number)` → Estimated delivery date
- `mark_delivered(order_id, signature_proof)` → Marks order as delivered
- `handle_failed_delivery(order_id, reason)` → Redelivery, return-to-sender logic

**Decision Logic**

- Dispatch within 3-5 business days (after confirmed payment)
- Choose courier based on destination, package weight, urgency
- Don't ship to unserviceable PIN codes
- Notify customer with tracking link immediately
- Failed delivery: try once more; if fails, initiate return process
- Signature required for orders over ₹5,000

**Escalation**

- Package delayed beyond estimate (alert Support Agent)
- Customer requests expedited shipping (pass to Admin Agent)
- Lost or severely damaged in transit (pass to Returns Agent)

## 6. Returns Agent

**Responsibility**: Exchange requests, refund logic, defective/incorrect item handling, legal remedies.

**Tools**

- `create_return_request(order_id, reason, customer_notes)` → Initiates return workflow

- `assess_return_eligibility(order_id, product_id, days_since_delivery)` → Checks policy, condition, window
- `approve_exchange(order_id, replacement_sku)` → Authorizes exchange, arranges reverse shipping
- `approve_refund(order_id, amount, reason)` → Triggers refund via Payment Agent
- `escalate_to_legal_remedy(order_id, issue_type)` → Flags defective/incorrect items for manual review

**Decision Logic**

- Standard policy: Exchange only within 7 days, unused/unworn/original tags
- Defective or incorrect items: Full refund or replacement regardless of window
- Damaged in shipping: Full refund or replacement
- Customer-caused damage: Evaluate case-by-case; legal remedies override policy
- Out-of-stock replacement: Offer refund or wait for restock
- Reverse shipping cost: Free for store-caused issues, customer pays for change-of-mind (if policy allows)

**Escalation**

- Complex return (multiple items, unclear responsibility): pass to Compliance Agent
- Returns outside window claiming defect: escalate to legal review
- Abusive return patterns: flag for Admin Agent investigation

## 7. Support Agent

**Responsibility**: Customer inquiries, problem-solving, escalation triage, knowledge base.

**Tools**

- `search_knowledge_base(query)` → FAQs, size guides, care instructions, policies
- `create_support_ticket(customer_id, issue_type, description)` → Opens ticket with auto-assignment
- `get_customer_history(customer_id)` → Order history, past interactions, VIP status
- `send_message(customer_id, message, channel)` → Email, SMS, or in-app notification
- `escalate_ticket(ticket_id, priority, agent_type)` → Routes to specialist (Returns, Payment, Compliance)

**Decision Logic**

- Try to resolve with knowledge base first
- For order-related questions, pull customer's order history
- For sizing issues, refer to size guide
- For policy questions, cite exact policy language
- For legal or complex issues, escalate immediately
- Response SLA: 4 hours for urgent, 24 hours for standard

**Escalation**

- Returns/exchanges: pass to Returns Agent
- Payment issues: pass to Payment Agent
- Policy exceptions or legal questions: pass to Compliance Agent
- Complaints requiring compensation: pass to Admin Agent

## 8. Compliance Agent

**Responsibility**: Policy enforcement, tax compliance, data privacy, legal review, publication gates.

**Tools**

- `validate_against_policy(policy_type, content)` → Checks if content violates policy
- `calculate_gst(product_sku, sale_price)` → Returns applicable GST amount
- `validate_pii_handling(data_operation)` → Ensures DPDP Act compliance
- `review_customer_communication(message, channel)` → Checks for misleading claims
- `approve_for_publication(content_type, content)` → Gating for website, ads, emails

**Decision Logic**

- Block claims without evidence ("handcrafted," "premium fabrics," "sustainable" without cert)
- Calculate GST per product: Clothing 5%, accessories vary
- Enforce DPDP Act: minimal data collection, explicit consent, retention limits
- Flag misleading discounts ("50% off" only if true reference price exists)
- Require approval for new policies before publication
- Block exports of customer data without audit

- Enforce returns/refunds per Consumer Protection Act

**Escalation**

- Potential tax violation: alert Admin Agent for manual review
- Privacy incident or data breach: escalate to security team immediately
- Legal question: pass to external counsel, not automated

## 9. Admin Agent

**Responsibility**: Configuration management, business rule updates, manual order handling, reporting, human-in-the-loop decisions.

**Tools**

- `update_business_config(key, value)` → Changes policies, thresholds, settings
- `create_manual_order(details)` → Handles phone/in-store orders
- `override_decision(entity_id, agent_type, original_decision, override_reason)` → Approves exceptions
- `get_metrics(metric_type, date_range)` → Orders, revenue, returns, agent decisions, satisfaction
- `export_data(entity_type, filters)` → Customer, order, product data export
- `generate_report(report_type)` → Inventory, sales, compliance, agent performance

**Decision Logic**

- Only authorized admins can update policies
- Manual orders created with same validations as online
- Overrides require reason, logged in audit
- Metrics dashboards show agent success rates, escalation rates, customer satisfaction
- Data exports require audit logging and are limited to necessary fields

**Escalation**

- Security concerns: escalate to security team
- Fraud patterns: escalate to law enforcement if needed

# Core Business Workflows

## Workflow 1: Product Discovery & Shopping

**Actor**: Customer **Agents Involved**: Catalog Agent, Shopping Agent, Support Agent

**Flow**:

1. Customer visits Noir Atelier website; Intent Classifier routes to Catalog Agent
2. Catalog Agent receives query: "Black shirts for office, size M, under ₹1,500"
3. Catalog Agent calls `search_products` with filters; returns 12 results with images, sizes, prices
4. Customer clicks on "Noir Essential Oversized Tee" (₹1,199)
5. Catalog Agent calls `get_product_details`; returns colors (black, navy), sizes (S-XXL), care instructions, reviews
6. Customer adds size M to cart; Shopping Agent validates availability via `check_inventory`
7. Shopping Agent displays cart: 1x Noir Essential Tee ₹1,199; subtotal ₹1,199 (eligible for free shipping at ₹1,999, but below threshold)
8. Shopping Agent offers related items ("Complete your look"): womens version, accessories
9. Customer browses more; Catalog Agent recommends "Signature Linen-Blend Shirt" (₹2,499) based on style
10. Customer asks about size fit; Support Agent provides size guide via `search_knowledge_base`

**Escalations**:

- If product out of stock: Catalog Agent suggests alternatives or waitlist
- If customer asks about material sourcing: Escalate to Support Agent (may need Compliance review if making sustainability claims)

## Workflow 2: Checkout & Payment

**Actor**: Customer **Agents Involved**: Shopping Agent, Order Agent, Payment Agent

**Flow**:

1. Customer proceeds to checkout with cart: 1x Noir Essential Tee (₹1,199) + 1x Linen-Blend Shirt (₹2,499) = ₹3,698

2. Shopping Agent validates cart: both items in stock, prices current
3. Shopping Agent calculates totals:
   - Subtotal: ₹3,698
   - GST (5% for clothing): ₹184.90
   - Free shipping (above ₹1,999 threshold)
   - **Total: ₹3,882.90**
4. Order Agent requests delivery address; validates PIN code (must be in serviceable list)
5. Order Agent presents payment options: UPI, Card, Net Banking, Bank Transfer
6. Customer selects UPI; Payment Agent calls `initiate_payment`
7. Payment gateway redirects customer to UPI payment; customer confirms via phone
8. Payment Agent receives webhook: payment confirmed for ₹3,882.90
9. Order Agent creates order record: Order #NA-20261029-00142, status = "Confirmed"
10. Order Agent triggers confirmation email with order summary, tracking info (will be available after dispatch)

**Escalations**:

- Payment failed: Payment Agent retries once; if fails again, asks customer for alternate method
- Address unserviceable: Order Agent blocks checkout; suggests alternate destination or informs customer we don't ship there
- GST calculation incorrect: Compliance Agent reviews and corrects

## Workflow 3: Order Fulfillment & Tracking

**Actor**: Fulfillment team + Customer **Agents Involved**: Fulfillment Agent, Order Agent, Support Agent

**Flow**:

1. Order created at 3:45 PM on Day 1 (Monday)
2. Fulfillment team picks and packs by Day 2 (Tuesday) afternoon
3. Fulfillment Agent calls `create_shipment` with order items; books Shiprocket for delivery to Bangalore
4. Fulfillment Agent receives AWB: SR123456789
5. Fulfillment Agent updates order, sends tracking email to customer: "Your order is on its way! Track: [link]"

6. Customer clicks tracking link; Fulfillment Agent calls `get_tracking`; shows real-time status
7. Day 3-4: Package in transit; Fulfillment Agent polls tracking every 4 hours for updates
8. Day 5 (Friday) 2:30 PM: Package delivered; Fulfillment Agent marks order as delivered, sends delivery confirmation email
9. Customer requests tracking update; Support Agent calls `get_delivery_estimate`; tells customer it was delivered on Day 5

**Escalations**:

- Delivery delayed > 7 days: Support Agent proactively alerts customer; escalates to Fulfillment Agent to investigate
- Failed delivery (address issue, recipient unavailable): Fulfillment Agent attempts redelivery; if fails again, initiates return to warehouse
- Customer reports damaged package: Escalate to Returns Agent

### Workflow 4: Returns & Exchanges

**Actor**: Customer **Agents Involved**: Returns Agent, Support Agent, Fulfillment Agent, Compliance Agent

**Flow**:

1. Customer orders on Day 1; receives Day 5. Day 6: Customer realizes size doesn't fit, initiates return
2. Support Agent receives request; calls `create_support_ticket`; triages to Returns Agent
3. Returns Agent checks order: placed 5 days ago, still within 7-day exchange window
4. Returns Agent calls `assess_return_eligibility`: Item unworn, original tags, within window = Eligible
5. Returns Agent offers: "Exchange for size L, or full refund"
6. Customer chooses exchange for size L
7. Returns Agent checks inventory for size L in black: 5 in stock
8. Returns Agent calls `approve_exchange`; authorizes reverse shipping (free for store)
9. Returns Agent arranges pickup: Fulfillment Agent books reverse shipping for Day 7
10. Customer packages item, gives to courier Day 7; Fulfillment Agent tracks return
11. Return received at warehouse Day 10; Returns Agent inspects: condition good, unused

12. Returns Agent calls Fulfillment Agent to send replacement (size L) immediately (priority dispatch)
13. Replacement dispatched Day 10 PM; delivered Day 12
14. Customer receives replacement, satisfied

**Escalations**:

- Customer outside 7-day window but claims defective: Escalate to Compliance Agent for legal review
- Replacement out of stock: Returns Agent offers refund or wait for restock
- Damage assessment disputed: Escalate to Admin Agent for manual review

## Workflow 5: Customer Support & Complaint Resolution

**Actor**: Customer **Agents Involved**: Support Agent, Order Agent, Returns Agent, Compliance Agent

**Flow**:

1. Customer calls/emails: "I've been waiting 10 days for my order. It says it was delivered, but I never received it."
2. Support Agent calls `get_customer_history`; finds order #NA-20261029-00142, delivery status = delivered Day 5
3. Support Agent calls Fulfillment Agent's `get_tracking`; confirms delivery signature obtained
4. Support Agent messages customer: "Our records show delivery on Day 5 at 2:30 PM to your address. Can you confirm if you received it? If not, we'll investigate with the courier."
5. Customer responds: "I was out. Let me check with my building security."
6. 2 hours later, customer confirms: "Found it! My building security received it. Apologies!"
7. Support Agent closes ticket with note: "Package was delivered, found by customer."

**Escalation scenario** (if unresolved):

1. Customer insists never received, despite delivery proof
2. Support Agent escalates to Returns Agent: treat as lost-in-delivery
3. Returns Agent approves replacement or refund (customer's choice)
4. Fulfillment Agent ships replacement or Payment Agent processes refund

5. Compliance Agent logs incident for dispute resolution records

### Workflow 6: Compliance & Policy Violation

**Actor**: Admin **Agents Involved**: Compliance Agent, Admin Agent

**Flow**:

1. Marketing team wants to publish: "Noir Atelier Premium Handcrafted Collections — Ethically Sourced"
2. Compliance Agent calls `validate_against_policy`; flags: "handcrafted" and "ethically sourced" lack evidence
3. Compliance Agent blocks publication; sends feedback: "These claims require:
   - Evidence of handcrafting process (documentation, certifications)
   - Supply chain transparency or third-party ethics certification
   - Without these, claims are misleading under Consumer Protection Act"
4. Marketing revises: "Noir Atelier Curated Collections — Thoughtfully Designed Contemporary Fashion"
5. Compliance Agent re-reviews; approves for publication
6. Admin Agent publishes update to website

**Escalations**:

- Attempted publication of unverified health/safety claims: Block immediately; escalate to legal counsel
- Repeated policy violations from same user: Escalate to Admin for access review

## Security & Compliance

### Data Protection

#### PII Handling

- Customer data: Name, email, phone, address, payment method reference (not full card/UPI credentials)
- Collected: During registration, checkout, support interactions only
- Storage: Encrypted at rest (AES-256); TLS 1.3 in transit
- Retention: 7 years for tax/legal reasons, else 2 years; deletion workflow per DPDP Act
- Access: Role-based; only agents/users with legitimate need access

- Audit: Every access logged with timestamp, user, action, result

**Payment Security**

- Never store card numbers, UPI IDs, or CVV locally
- Use payment gateway tokenization (Razorpay, PayU)
- Webhook verification: cryptographic signature validation
- PCI DSS compliance: annual audits by third-party assessor

**Data Residency**

- Customer data stored in India (AWS Mumbai or local equivalent)
- Compliance: RBI guidelines, DPDP Act, GST records
- No cross-border transfer without explicit consent (and legal basis)

## Agent Security

**Tool Calling Constraints**

- Agents operate only on explicitly defined tools (schema-driven)
- No code execution; no SQL injection; no access to system files
- All tool calls logged with input, output, duration, result code
- Tool failures caught explicitly; never silently fail

**Privilege Escalation Prevention**

- Catalog/Shopping/Order Agents: read-only or limited writes
- Returns/Compliance/Admin Agents: restricted to authorized actions
- Refunds, policy changes, exports: require human approval (not automated)
- Audit log immutable after creation

**Prompt Injection Protection**

- Customer inputs (queries, reviews, messages) never directly injected into prompts
- All user input validated, sanitized, escaped
- Agent system prompts static and version-controlled
- Jailbreak attempts logged and escalated

## Compliance Frameworks

**Consumer Protection Act (India)**

- Mandatory disclosures: seller identity, price, delivery terms, return/refund policy
- Grievance officer contact published
- Response SLA: 48 hours for complaints
- No unilateral cancellation of orders or refunds without cause

**E-commerce Rules (India)**

- Terms and Conditions clearly displayed before purchase
- Returns policy enforced as stated; cannot deviate for financial reasons
- Customer consent for data collection (opt-in for marketing)
- Order confirmation via email/SMS within 7 days

**Data Protection (DPDP Act 2023)**

- Consent for data collection (already obtained)
- Data minimization: collect only what's needed
- Retention: delete after legitimate purpose ends
- User rights: access, correction, deletion requests honored within 30 days
- Data processor agreements for third parties (shipping, payment, analytics)

**GST Compliance**

- Tax calculation per HSN code; clothing is 5% (mostly)
- Invoices generated for all orders
- Tax remittance on schedule per GST rules
- Compliance Agent reviews all calculations quarterly

**Information Security**

- ISO 27001 certification (target)
- Annual penetration testing
- Bug bounty program for security researchers
- Incident response plan: detection, containment, remediation, disclosure (24 hours to customers if data affected)

## Fraud Detection & Prevention

**Order-Level Fraud**

- Payment Agent flags: new customer + high value + unusual delivery address
- Multiple orders in short time from same card

- Shipping address doesn't match billing address (review manually)
- Frequent returns (serial returners) flagged for manual inspection

**Account-Level Fraud**

- Admin Agent monitors: unusual login locations, rapid account creation, fake reviews
- CAPTCHA on suspicious login attempts
- Email verification for new accounts

**Operational Fraud**

- Staff audit: who accessed what, when, why
- Segregation of duties: no single person can authorize and execute refunds
- Random audit of high-value transactions (>₹50,000)
- Whistleblower channel for internal fraud reporting

# Observability, Monitoring & Debugging

## Logging Architecture

**Structured Logging**

- All agent decisions, tool calls, errors logged in JSON format
- Fields: timestamp, agent_id, tool_name, input_params, output, duration_ms, error, customer_id, order_id, session_id
- Logs shipped to Elasticsearch for search and analytics
- Retention: 90 days hot, 1 year cold storage (S3)

**Log Examples**

```
{
  "timestamp": "2026-10-29T17:59:00Z",
  "level": "info",
  "agent": "shopping_agent",
  "action": "apply_coupon",
  "coupon_code": "WELCOME10",
  "discount_amount": 380,
  "cart_total_before": 3800,
  "cart_total_after": 3420,
  "customer_id": "cust_12345",
  "session_id": "sess_abc123",
  "status": "success"
}
```

## Metrics & Dashboards

### Key Metrics

- **Order Volume**: Orders/day, average order value, conversion rate
- **Agent Performance**: Success rate per agent, average response time, error rate
- **Customer Satisfaction**: NPS, support ticket resolution rate, return rate
- **Business Health**: Revenue, returns %, refund rate, fraud incidents
- **System Health**: API latency, error rate, agent timeouts, tool call failures

**Dashboards** (Metabase or Grafana)

- Real-time: Current orders processing, live agent activity, support ticket queue
- Daily: Orders, revenue, returns, refunds, fraud flags, agent decisions
- Weekly: Trend analysis, customer satisfaction, system reliability
- Monthly: Compliance review, policy effectiveness, scaling metrics

## Tracing & Debugging

### Distributed Tracing

- Every customer request gets a unique trace_id
- Trace flows through: API Gateway → Intent Classifier → Agent(s) → Tool Calls → Database
- Each step logs: timestamp, service, duration, status, trace_id
- Enables root-cause analysis: "Why did this refund fail?"

### Debugging Tools

- Agent reasoning logs: thought process, intermediate decisions, tool call reasons
- Tool execution logs: what was called, with what params, what was returned
- Error traces: full stack, context, recovery attempts
- Customer session replay: replay agent interactions for a specific order/session

## Alerting

**Critical Alerts** (immediate)

- Payment processing failure rate > 5%
- Refund processing stuck > 30 min
- Database unavailable
- Agent crash or timeout
- Security incident detected

**Warning Alerts** (within 1 hour)

- High return rate (>10% daily)
- Compliance violation attempt
- Agent error rate > 2%
- Unusual fraud pattern detected

**Info Alerts** (daily summary)

- Orders processed, revenue, system health score
- Top support issues, common failures
- Agent performance trends

## Continuous Monitoring

### Weekly Reviews

- Agent success/failure breakdown
- Customer pain points (from support tickets)
- System performance trends
- Compliance findings

### Monthly Reviews

- Agent reasoning quality (sample orders, review decisions)

- Policy effectiveness (are policies working as intended?)
- Fraud patterns and prevention effectiveness
- Scaling readiness (can system handle 2x volume?)

**Quarterly Reviews**

- Agent prompt engineering evaluation and updates
- Tool schema improvements
- Integration reliability (payment, shipping, tax)
- Security audit results

# Deployment Strategy

## Environment Configuration

**Development (Local + Docker Compose)**

- Mock data: seed products, customers, orders
- Simulated payments: no real charges
- Local PostgreSQL + Redis + Elasticsearch
- Agent reasoning visible in debug logs
- Fast iteration: deploy on every commit

**Staging (AWS/GCP, Production-Like)**

- Real database (isolated from production)
- Real integrations: test payment credentials, test shipping APIs, test tax calculations
- Full agent reasoning logs; no production customers
- A/B testing environment for new agent prompts
- Weekly full system test; data reset after each test

**Production (Kubernetes, Multi-Region)**

- Real data: live customers, actual orders, real payments
- High availability: 3+ replicas per service
- Load balancing: round-robin, health checks
- Auto-scaling: CPU/memory triggers
- Zero-downtime deployments: rolling updates
- Disaster recovery: automated backups, cross-region failover

## Deployment Pipeline

```
Developer Push to main
    ↓
GitHub Actions: Run tests, lint, security scan
    ↓
Build Docker images, push to ECR/GCR
    ↓
Deploy to Staging (automatic)
    ↓
Smoke tests: health checks, API endpoints, agent reasoning
    ↓
Manual approval (for production) by tech lead
    ↓
Deploy to Production (rolling: 10% → 50% → 100%)
    ↓
Monitoring: error rate, latency, customer complaints
    ↓
Automatic rollback if error rate > 5% in first 5 min
```

## Configuration Management

### Environment Variables

- Secrets (API keys, DB passwords): AWS Secrets Manager / HashiCorp Vault
- Feature flags: LaunchDarkly or Unleash (enable/disable agents, tools, features per environment)
- Business config (free shipping threshold, exchange window, GST rates): ConfigMap in Kubernetes
- Policy documents: stored in database, versioned, published via CMS

### Database Migrations

- Version controlled in Git (Alembic for SQLAlchemy, Flyway for raw SQL)
- Always reversible (down migrations)
- Tested on staging before production
- Zero-downtime: add column nullable, backfill, add constraint

## Scaling Strategy

### Horizontal Scaling

- Stateless services: API servers, agent workers, background jobs
- Kubernetes HPA (Horizontal Pod Autoscaler): CPU > 70% → scale up
- Database: read replicas for analytics; write to primary only
- Message queue: partition by order_id for ordering guarantees

**Vertical Scaling**

- Agent complexity (reasoning time) grows slowly; monitor via metrics
- Tool call latency: cache frequent queries (product search, inventory)
- Database indexes: monitor slow queries, add indexes as needed

**Caching Strategy**

- Redis: product data, customer session, cart state (1-day TTL)
- CDN: product images, static assets (CloudFront, Cloudflare)
- Vector DB: product embeddings for semantic search
- Database query cache: via pgBouncer (connection pooling)

# Implementation Roadmap

## Phase 1: Foundation (Weeks 1-6) — MVP Core

**Deliverables**

- GitHub repo with project structure (FastAPI backend, Next.js frontend, Docker setup)
- PostgreSQL schema: customers, products, orders, payments, support tickets
- Basic Catalog Agent: product search, filters, recommendations
- Basic Order Agent: cart, checkout, order creation
- Mock payment processing (no real charges)
- Admin dashboard: view orders, products, basic metrics

**Success Criteria**

- 5 demo products in catalog
- Customer can browse, add to cart, checkout (order created but not paid)
- Admin can view orders and products
- All code has unit tests (80%+ coverage)

**Team**

- 1 backend engineer (API, agents, database)
- 1 frontend engineer (UI, cart, checkout)
- 1 ML engineer (embeddings, semantic search)

## Phase 2: Monetization (Weeks 7-12) — Payments & Fulfillment

**Deliverables**

- Payment Agent integration: Razorpay or PayU (test mode)
- Payment reconciliation: webhook verification, order confirmation
- Fulfillment Agent integration: Shiprocket API (test shipping)
- Tracking updates: customer notifications via email/SMS
- Returns Agent: basic exchange logic (7-day window)
- Support Agent skeleton: ticket creation, knowledge base

**Success Criteria**

- End-to-end order: add to cart → pay → shipped → delivered
- Payment verified within 5 minutes of order
- Tracking accessible to customer
- Returns initiated, processed, replacement shipped

**Team**

- 1 backend engineer (payment, fulfillment integrations)
- 1 payments specialist (PCI compliance, webhook testing)
- 1 QA engineer (end-to-end testing)

## Phase 3: Compliance & Safety (Weeks 13-18) — Legal, Privacy, Fraud

**Deliverables**

- Compliance Agent: policy validation, GST calculation, legal checks
- Privacy implementation: DPDP Act compliance, consent forms, data deletion
- Security hardening: input validation, prompt injection prevention, RBAC
- Fraud detection: order flagging, velocity checks
- Audit logging: all agent decisions, tool calls logged
- Legal review: policies drafted, terms approved by counsel

**Success Criteria**

- No unverified claims on website
- GST correctly calculated per product
- Customer can request data deletion and receive confirmation
- Suspicious orders flagged for manual review
- All decisions (approvals, refunds, escalations) logged

**Team**

- 1 backend engineer (compliance agent, logging)
- 1 security engineer (encryption, audit, RBAC)
- 1 legal consultant (policy review, compliance)

## Phase 4: Scaling & Reliability (Weeks 19-24) — Production Ready

**Deliverables**

- Kubernetes deployment: dev, staging, production environments
- CI/CD pipeline: GitHub Actions, automated testing, deployment
- Observability: Datadog/New Relic, dashboards, alerting
- High availability: load balancing, database replication, failover
- Performance optimization: caching, query optimization, async jobs
- Load testing: simulate 1000 concurrent users, 100 orders/min
- Disaster recovery: automated backups, cross-region failover plan

**Success Criteria**

- Deployment to production in < 5 minutes
- 99.9% uptime during peak hours
- P95 latency < 500ms for customer requests
- Auto-scaling triggered and functional
- Full data backup and restore tested

**Team**

- 1 DevOps/SRE engineer (infrastructure, CI/CD, monitoring)
- 1 backend engineer (performance optimization)
- 1 QA engineer (load testing, chaos engineering)

## Phase 5: Advanced Features (Weeks 25-36) — Intelligence & Growth

### Deliverables

- Personalization: customer preference learning, recommendation improvements
- Multi-language support: Hindi, Tamil, Telugu (relevant to Tamil Nadu)
- Advanced support: AI-powered responses, sentiment analysis, escalation prediction
- Inventory optimization: demand forecasting, reorder suggestions
- Analytics dashboards: customer lifetime value, churn prediction, revenue forecasts
- A/B testing framework: test new agent prompts, UI changes
- Admin tools: manual overrides, bulk operations, custom reports

### Success Criteria

- Recommendation click-through rate > 5%
- Support resolution without escalation > 70%
- Inventory stockouts < 5%
- A/B tests running for new features

### Team

- 1 ML engineer (personalization, forecasting)
- 1 backend engineer (new features, optimization)
- 1 data analyst (analytics, dashboards)

## Critical Path & Dependencies

```
Phase 1: Foundation        |--|  (6 weeks)
  Phase 2: Payments       |--------| (6 weeks)
    Phase 3: Compliance   |--------| (6 weeks)
      Phase 4: Scaling    |--------| (6 weeks)
        Phase 5: Advanced |--------| (12 weeks)
```

- Phase 1 must complete before Phase 2 (need orders before payments)
- Phase 3 can start after Phase 2 starts (parallel work on compliance)
- Phase 4 should start by end of Phase 3 (need infrastructure before production)
- Phase 5 starts after Phase 4 (need stable foundation first)

## Resource Estimate

- **Total duration**: 36 weeks (~8-9 months)
- **Peak team**: 8-10 engineers, 1 consultant
- **Infrastructure cost**: ~₹20,000/month (development) + ₹100,000+/month (production)
- **Third-party costs**: Razorpay (2% transaction), Shiprocket (₹15-50/shipment), Datadog (enterprise tier), Twilio (SMS)

## Risks & Mitigations

| Risk | Impact | Mitigation |
|---|---|---|
| Payment integration delays | Phase 2 blocked | Use test mode early; parallel sandbox testing |
| GST classification unclear | Compliance issue | Engage tax professional in Phase 1; test with sample products |
| Courier unavailability | Fulfillment risk | Integrate 2+ courier partners by Phase 2 |
| Agent reasoning quality | Customer frustration | Prompt engineering, eval framework, A/B testing |
| Data privacy audit failure | Legal/financial risk | Compliance consultant in Phase 3; external audit in Phase 4 |
| Database scaling limits | Performance degradation | Load testing in Phase 4; read replicas, sharding plan ready |

## Post-Launch (Months 10+) — Operations & Evolution

- **Week 1 after launch**: Monitor 24/7; incident response team on-call
- **Week 2-4**: Fix bugs, optimize performance, respond to customer feedback
- **Month 2+**: Feature roadmap based on customer requests, agent improvement cycle
- **Quarterly**: Compliance review, security audit, competitive analysis, expansion planning

## Success Metrics

- **Functional**: All core agents working, zero critical bugs in production
- **Business**: 100+ orders/week, ₹75K+ revenue/month, 90%+ customer satisfaction

- **Operational**: 99.9% uptime, sub-500ms P95 latency, < 1 hour to resolution for critical bugs
- **Compliance**: Zero regulatory violations, all customer data secure, audit pass